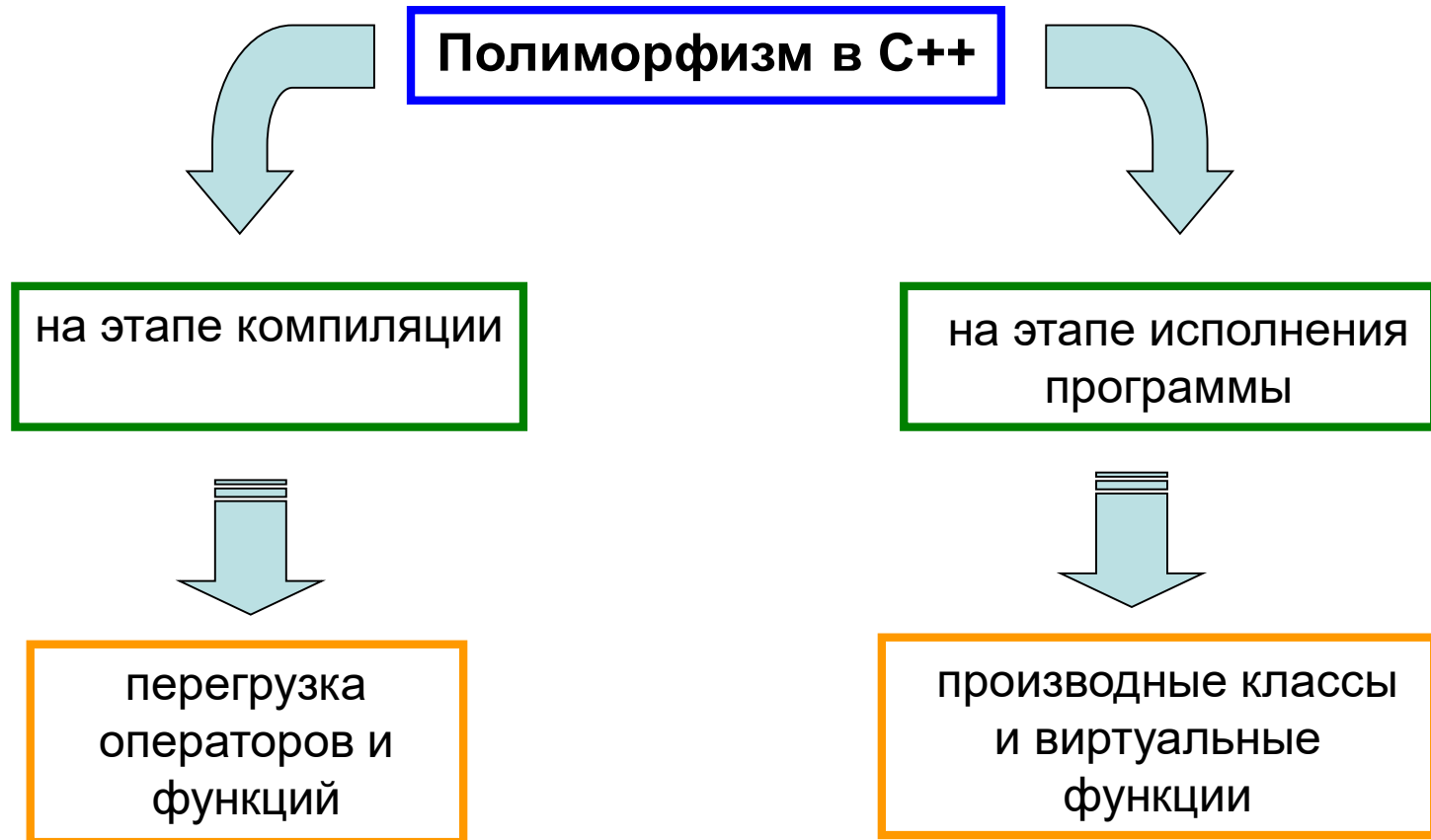


Лекция 7

Полиморфизм в языке программирования C++

1 Полиморфизм в C++



2 Понятие виртуальной функции

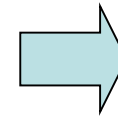
Виртуальные функции – это функции, которые объявляются с использованием ключевого слова `virtual` в базовом классе и переопределяются в одном или нескольких производных классах.

При вызове функции, объявленной виртуальной, через указатель на базовый тип, во время выполнения программы определяется, какая виртуальная функция будет вызвана, в зависимости от того, на объект какого класса будет указывать указатель.

В примере не используется
виртуальная функция

```
1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6  public:
7      void Show()
8      { cout << "Родитель" << endl; }
9  };
10
11  class ChildA : public Parent
12  {
13  public:
14      void Show()
15      { cout << "Потомок А" << endl; }
16  };
17
18  class ChildB : public Parent
19  {
20  public:
21      void Show()
22      { cout << "Потомок В" << endl; }
23  };
24
```

```
25  int main()
26  {
27      ChildA A;
28      ChildB B;
29      Parent *P;
30
31      P = &A;
32      P->Show();
33
34      P = &B;
35      P->Show();
36
37      A.Show();
38      B.Show();
39  }
```



Родитель
Родитель
Потомок А
Потомок В

```

1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6  public:
7  virtual void Show() // ВИРТУАЛЬНЫЙ МЕТОД!!!
8  { cout << "Родитель" << endl; }
9  };
10
11 class ChildA : public Parent
12 {
13 public:
14 void Show()
15 { cout << "Потомок А" << endl; }
16 };
17
18 class ChildB : public Parent
19 {
20 public:
21 void Show()
22 { cout << "Потомок В" << endl; }
23 };
24

```

Для программы на
предыдущем слайде
изменена одна строка
(строка 7) – метод Show()
теперь виртуальный!!!

```

25 int main()
26 {
27     ChildA A;
28     ChildB B;
29     Parent *P;
30
31     P = &A;
32     P->Show();
33
34     P = &B;
35     P->Show();
36
37     A.Show();
38     B.Show();
39 }

```

Потомок А
Потомок В
Потомок А
Потомок В

3 Абстрактные классы и чистые виртуальные функции

Базовый класс, объекты которого никогда не будут реализованы, называется *абстрактным классом*.

Чистая виртуальная функция – это функция, после объявления которой добавлено выражение $= 0$ (для защиты объектов базового класса от реализации)

Метод Show() ЧИСТЫЙ
виртуальный (строка 7), а класс
Parent абстрактный и объекты
класса Parent нельзя создавать
(строка 30)!!!

```
1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6  public:
7      virtual void Show() = 0; // ЧИСТЫЙ ВИРТУАЛЬНЫЙ МЕТОД!!!
8  };
9
10 class ChildA : public Parent
11 {
12 public:
13     void Show()
14     { cout << "Потомок А" << endl; }
15 };
16
17 class ChildB : public Parent
18 {
19 public:
20     void Show()
21     { cout << "Потомок В" << endl; }
22 };
23
```

```
24 int main()
25 {
26     ChildA A;
27     ChildB B;
28     Parent *P;
29
30     // Parent P1; // ОШИБКА!!!
31
32     P = &A;
33     P->Show();
34
35     P = &B;
36     P->Show();
37
38     A.Show();
39     B.Show();
40 }
```

Потомок А
Потомок В
Потомок А
Потомок В

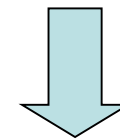
4 Виртуальные деструкторы

Деструктор базового класса обязательно должен быть виртуальным!!!

Пример программы без виртуального деструктора:

```
1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6  public:
7      ~Parent()
8      { cout << "Сработал деструктор "
9          << "класса Parent" << endl; }
10 };
11
12 class ChildA : public Parent
13 {
14 public:
15     ~ChildA()
16     { cout << "Сработал деструктор "
17         << "класса ChildA" << endl; }
18 };
19
20 class ChildB : public Parent
21 {
22 public:
23     ~ChildB()
24     { cout << "Сработал деструктор "
25         << "класса ChildB" << endl; }
26 };
```

```
27
28 int main()
29 {
30     Parent *P = new Parent;
31     Parent *PA = new ChildA;
32     Parent *PB = new ChildB;
33
34     delete P;
35     delete PA;
36     delete PB;
37 }
```

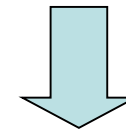


```
Сработал деструктор класса Parent
Сработал деструктор класса Parent
Сработал деструктор класса Parent
```


Пример программы с виртуальным деструктором:

```
1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6  public:
7      virtual ~Parent()
8  { cout << "Сработал деструктор "
9      << "класса Parent" << endl; }
10 };
11
12 class ChildA : public Parent
13 {
14 public:
15     ~ChildA()
16 { cout << "Сработал деструктор "
17     << "класса ChildA" << endl; }
18 };
19
20 class ChildB : public Parent
21 {
22 public:
23     ~ChildB()
24 { cout << "Сработал деструктор "
25     << "класса ChildB" << endl; }
26 };
```

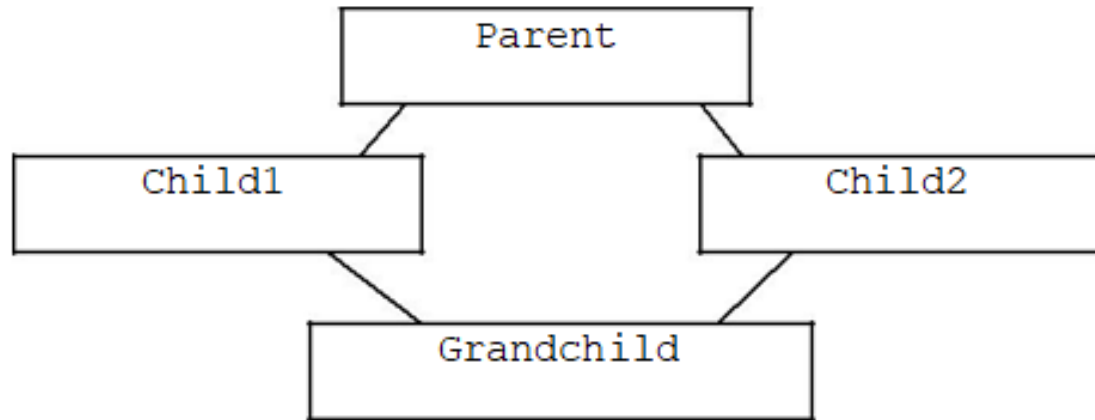
```
27
28 int main()
29 {
30     Parent *P = new Parent;
31     Parent *PA = new ChildA;
32     Parent *PB = new ChildB;
33
34     delete P;
35     delete PA;
36     delete PB;
37 }
```



```
Сработал деструктор класса Parent
Сработал деструктор класса ChildA
Сработал деструктор класса Parent
Сработал деструктор класса ChildB
Сработал деструктор класса Parent
```

5 Виртуальные базовые классы

Виртуальный базовый класс — это класс, объект которого является общим для использования всеми дочерними классами



```
class Parent
{ ... };
class Child1: virtual public Parent
{ ... };
class Child2: virtual public Parent
{ ... };
class Grandchild: public Child1, public Child2
{ ... };
```

При создании класса Grandchild, мы получим только одну копию Parent, которая будет общей как для Child1, так и для Child2.

```

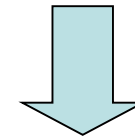
1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6  public:
7      Parent(int P)
8      { cout << "Parent: " << P << '\n'; }
9  };
10
11 class Child1: virtual public Parent
12 // Parent теперь виртуальный базовый класс
13 {
14 public:
15     Child1(int P, int Ch1):
16     Parent(P) // игнорируется
17     { cout << "Child1: " << Ch1 << '\n'; }
18 };
19
20 class Child2: virtual public Parent
21 {
22 public:
23     Child2(int P, int Ch2):
24     Parent(P) // игнорируется
25     { cout << "Child2: " << Ch2 << '\n'; }
26 };

```

```

27
28 class Grandchild: public Child1, public Child2
29 {
30 public:
31     Grandchild(int P, int Ch1, int Ch2):
32         Parent(P), // выполняется
33         Child1(P, Ch1), Child2(P, Ch2) { }
34 };
35
36 int main()
37 {
38     Grandchild GCh(1, 2, 3);
39 }

```



```

Parent: 1
Child1: 2
Child2: 3

```

6 Массив объектов родительского класса с объектами производных классов

Пример №1:

```
1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6  public:
7      virtual void Show()
8      { cout << "Класс Parent" << endl; }
9  };
10
11 class ChildA : public Parent
12 {
13 public:
14     void Show()
15     { cout << "Класс ChildA" << endl; }
16 };
17
18 class ChildB : public Parent
19 {
20 public:
21     void Show()
22     { cout << "Класс ChildB" << endl; }
23 };
```

```
24
25 int main()
26 {
27     // массив указателей на базовый класс
28     Parent *Mas[5];
29
30     // заполнение массива
31     Mas[0] = new ChildA;
32     Mas[1] = new ChildB;
33     Mas[2] = new Parent;
34     Mas[3] = new ChildB;
35     Mas[4] = new ChildA;
36
37     // вызов метода Show() для объектов массива
38     for(int i=0; i<5; i++)
39     { Mas[i]->Show(); }
40
41 }
```

```
Класс ChildA
Класс ChildB
Класс Parent
Класс ChildB
Класс ChildA
```

Пример №2:

```
1  #include <iostream>
2  using namespace std;
3
4  class Parent
5  {
6      protected:
7          char P;
8      public:
9          Parent(char p): P(p) {}
10         virtual void Show() = 0;
11         char GetP() { return P; }
12 };
13
14 class ChildA : public Parent
15 {
16     int A;
17     public:
18     ChildA(char p, int a): Parent(p), A(a) {}
19     void Show() { cout << "Класс ChildA : P = " << P << ", A = " << A << endl; }
20     void ShowA() { cout << "A = " << A << endl; }
21 };
22
23 class ChildB : public Parent
24 {
25     int B1;
26     int B2;
27     public:
28     ChildB(char p, int b1, int b2): Parent(p), B1(b1), B2(b2) {}
29     void Show() { cout << "Класс ChildB : P = " << P << ", B1 = " << B1 << ", B2 = " << B2 << endl; }
30     void ShowB() { cout << "B1 = " << B1 << ", B2 = " << B2 << endl; }
31 };
```



```

33 int main()
34 {
35     Parent *Mas[5]; // массив указателей на базовый класс
36
37     // заполнение массива
38     Mas[0] = new ChildA('A',1);
39     Mas[1] = new ChildB('B',2,2);
40     // Mas[2] = new Parent('A'); // ошибка!
41     Mas[2] = new ChildA('A',3);
42     Mas[3] = new ChildB('B',4,4);
43     Mas[4] = new ChildA('A',5);
44
45     // вызов метода Show() для объектов
46     for(int i=0; i<5; i++) { Mas[i]->Show(); }
47
48     // для объектов класса ChildA вызов метода ShowA()
49     cout << endl << "Вызов метода ShowA():" << endl;
50     for(int i=0; i<5; i++)
51     {
52         if (Mas[i]->GetP() == 'A')
53         {
54             // Mas[i]->ShowA(); // ошибка, метода ShowA()
55             // для Parent не существует
56             dynamic_cast<ChildA*>(Mas[i])->ShowA();
57         }
58     }
59     // для объектов класса ChildB вызов метода ShowB()
60     cout << endl << "Вызов метода ShowB():" << endl;
61     for(int i=0; i<5; i++)
62     {
63         if (Mas[i]->GetP() == 'B')
64             { dynamic_cast<ChildB*>(Mas[i])->ShowB(); }
65     }
66 }

```

```

Класс ChildA : P = A, A = 1
Класс ChildB : P = B, B1 = 2, B2 = 2
Класс ChildA : P = A, A = 3
Класс ChildB : P = B, B1 = 4, B2 = 4
Класс ChildA : P = A, A = 5

```

Вызов метода ShowA() :

```

A = 1
A = 3
A = 5

```

Вызов метода ShowB() :

```

B1 = 2, B2 = 2
B1 = 4, B2 = 4

```